

CopperForge Functional Description Document

A Gerber-First PCB Viewer and CAM Workbench

Prepared by

James Bonanno

<james@atlantixeng.com>

Git Version: unknown

Last Updated: 2026-04-22

Abstract

CopperForge is a gerber-first PCB viewer and CAM workbench. It reads the universal fab-interchange format — gerber files — as its primary geometry source, then drives both the 2D canvas view and the 3D extruded view from that one source of truth. This decouples the tool from any single EDA vendor: the same pipeline that loads a KiCad export will load an Altium, Eagle, or EasyEDA export unchanged, because the input is always gerbers.

- **Gerber as primary input** — PCB design files (`.kicad_pcb` , etc.) are treated as convenient starting points that the tool can convert to gerbers via the EDA vendor's CLI. Once gerbers exist on disk, every downstream pipeline stage reads from them, not the source PCB file.
- **2D geometry as intermediate representation** — the tessellated polygon data that drives the 2D canvas view is the same data the 3D extruder consumes. Rendering in two dimensions is not a separate pipeline; it is an intermediate stage of the 3D pipeline.
- **Fab truth** — because the 3D view is driven by gerbers, it shows what the fabrication house actually receives. Gerber-export bugs in the upstream EDA tool become visible in the 3D view, rather than being masked by a parallel PCB-file-direct parser.

This document specifies the CopperForge functional architecture, the gerber processing pipeline from file to rendered mesh, the 3D renderer structure, and the roadmap for extending the tool from its current KiCad-centric project workflow to true cross-vendor gerber loading.

Table of Contents

1. Motivation	5
1.1. What This Architecture Rejects	5
2. Gerber Processing Pipeline	7
2.1. Legacy 2D Rendering Path	8
3. Stage-by-Stage Reference	10
3.1. Stage 1: Gerber File Discovery	10
3.2. Stage 2: Gerber Document Parse	10
3.3. Stage 3: Geometry Extraction	10
3.4. Stage 4: Contour Stitching	11
3.5. Stage 5: Tessellation	12
3.6. Stage 6: World-Space Transform	12
3.7. Stage 7: GPU Upload and Render	12
4. Viewer 3D Features	13
4.1. Ground Grid	13
4.2. Zoom to Region	13
4.3. Flip	14
4.4. Measure	14
4.5. Hotkeys	14
4.6. 3D Gizmo	15
5. Phased Rollout	16
5.1. Phase 3 Scope (current work)	16
5.2. Deferred	16
6. References	17

Document History

Version	Description	Date	Author
0.1.0	Initial draft — gerber-direct pipeline architecture, block diagram, Phase 3 scope	22 Apr 2026	JB
0.2.0	Single-parse architecture: 2D via <code>gerber_viewer</code> re-framed as legacy bypass, not a parallel pipeline. Roadmap extended with explicit retirement phase.	22 Apr 2026	JB
0.3.0	Added Section 4 “Viewer 3D Features” documenting grid, zoom-to-region, flip, measure, hotkeys, and the 3D gizmo. Stage 6 updated: centered-at-origin world transform.	22 Apr 2026	JB
0.3.1	Hotkey routing via <code>egui_citizen</code> : 2D vs 3D tab activation gates keys like F/R/M so they fire in the correct view. “ <code>gerber_view_3d</code> ” citizen added to the registration list; the global dispatcher reads the one-hot active flag.	22 Apr 2026	JB

Scope

This document describes the functional architecture of *CopperForge*. It covers:

- The gerber-direct processing pipeline from file to rendered 3D mesh.
- The role of 2D gerber geometry as an intermediate stage of the 3D pipeline.
- Module boundaries between gerber parsing, geometry extraction, tessellation, and rendering.
- The phased rollout of the 3D viewer (tracked in `develop/task3d-plan.md`).
- Known limitations and planned cross-vendor extensions.

This document does *not* cover the existing KiCad integration (project database, `.kicad_pcb` layer name pull, DRC operations), which are orthogonal features that plug into the gerber pipeline rather than define it.

Purpose

The purpose of this document is to:

- Establish a single reference for the CopperForge gerber processing pipeline, so new work aligns with the documented data flow rather than ad-hoc decisions.
- Describe the separation between gerber file parsing, polygon extraction, tessellation, and GPU rendering — so each stage can be tested, swapped, or extended independently.
- Distinguish *CopperForge's architecture* (gerber-first, vendor-neutral) from *KiCad's 3D viewer* (PCB-file-direct, KiCad-specific), clarifying why the tool is built around the gerber pipeline.
- Surface open design decisions (dual parse vs. unified parse, drill file handling, cross-vendor gerber edge cases) for future resolution.

1. Motivation

The existing open-source EDA ecosystem pairs strong PCB design tools (KiCad, Horizon) with weak post-design inspection — no cross-vendor 3D viewer exists that operates on the universal fabrication interchange format. KiCad’s own 3D viewer is tightly coupled to the `.kicad_pcb` file and its OpenCASCADE component library; it cannot load gerbers, and it cannot show a non-KiCad board at all. At the same time, the gerber format is the single artifact every fabrication house accepts and every EDA tool produces. A gerber-first tool therefore occupies a genuinely empty category: the only cross-vendor PCB viewer driven by what the fab actually sees.

CopperForge takes the position that gerber files are the authoritative source of board geometry for any post-design workflow. Design files (`.kicad_pcb`, Altium `.PcbDoc`, etc.) are starting points convenient for automation, but the fabrication truth lives in the gerbers. A viewer that reads gerbers first, and only uses the design file for auxiliary metadata (project-scoped layer names, BOM hints), is automatically vendor-neutral — the same pipeline that renders a KiCad export renders any other EDA tool’s export.

This has two concrete consequences for the architecture:

1. **The gerber parser is the primary geometry source.** All rendered views — 2D canvas, 3D extrusion, future DRC overlays — read geometry that originates in a gerber file. The `.kicad_pcb` parser (for project metadata) is a secondary, optional data source.
2. **The 2D and 3D views share the same geometry pipeline.** Polygon extraction happens once per loaded project; the 2D canvas and the 3D extruder are downstream consumers of the same intermediate representation. This eliminates the class of bugs where the two views disagree.

1.1. What This Architecture Rejects

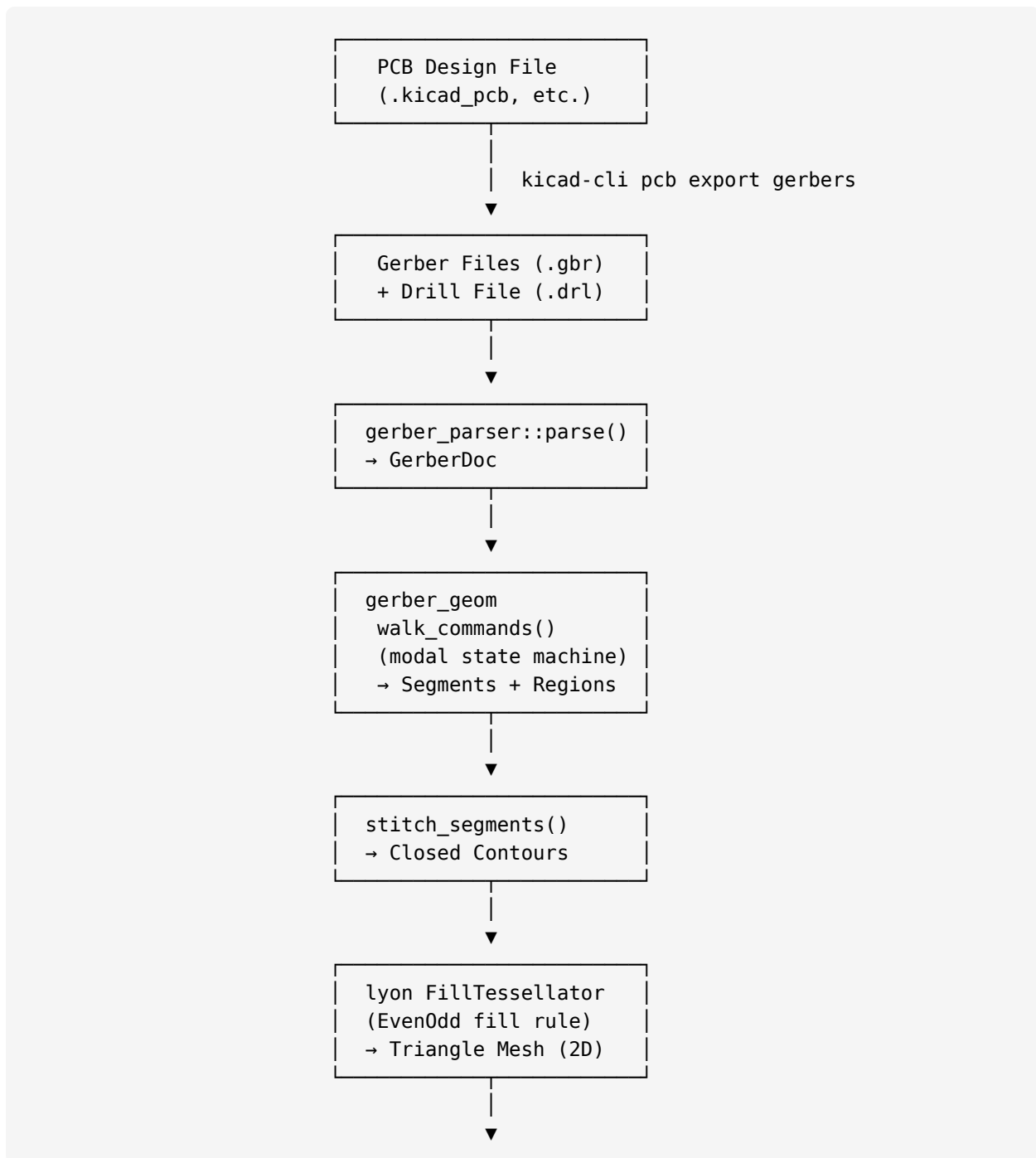
Earlier work on CopperForge’s 3D viewer (feature branch `render-3d`, archived) parsed the `.kicad_pcb` file directly — extracting `Edge.Cuts` primitives, footprint pads, via rings, and zone fills via s-expression traversal. That pipeline produced a 3D view but was:

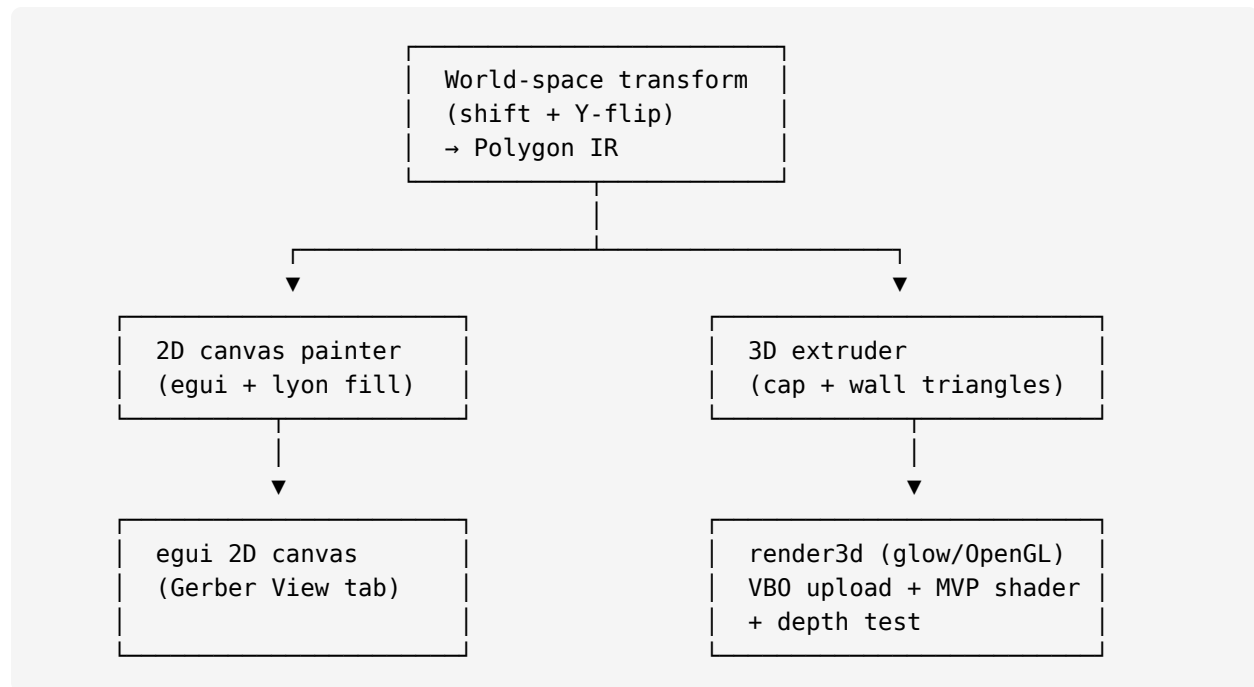
- **Vendor-locked** — only worked for KiCad sources. A board from any other EDA tool would have no 3D view at all.
- **Divergent from fab truth** — the 3D view showed the design file’s idea of the board, not what the gerber export contained. Export bugs (missing layers, clearance drift) were silently masked.
- **Duplicative of KiCad’s own 3D viewer** — reimplemented what KiCad already does, without the benefit of its component 3D models.

That branch is preserved in `~/Archives/copperforge-render3d-pcb-direct.bundle` for reference but is not part of the active architecture.

2. Gerber Processing Pipeline

CopperForge's pipeline is **single-parse by design**. Every gerber file is read exactly once, parsed once into a structured document, lowered once into a polygon intermediate representation, and then fanned out to every view that needs to render it. The 2D canvas and the 3D extrusion are both downstream consumers of this single polygon IR; they are not parallel pipelines.





Design Decision The polygon IR — closed contours + tessellated triangle mesh, in world coordinates with a known bbox — is the **shared product** of the pipeline. Every renderer that draws board geometry reads from it. This is a deliberate rejection of the alternative where each view owns its own parser: a single authoritative geometry source is what makes cross-vendor gerber loading tractable (one format, one parser, N renderers) and what guarantees that the 2D and 3D views cannot disagree about where a pad or trace is.

2.1. Legacy 2D Rendering Path

Today's 2D canvas does not yet consume the polygon IR. It uses `gerber_viewer::GerberRenderer::paint_layer()`, which wraps `gerber_parser::parse()` internally and returns an opaque `GerberLayer` whose primitive data is private (`pub(crate)`). This is **not** a parallel architectural path — it is a transitional bypass around the documented pipeline, retained because the 2D canvas works today and there is no functional gap to fix.

Two facts worth calling out:

1. **Parsing is not duplicated as a design choice.** `gerber_viewer` transitively depends on `gerber_parser`; so does `gerber_geom`. When both are wired up, each gerber file is read and parsed twice — once inside `gerber_viewer` for the 2D path, once directly by `gerber_geom` for the 3D path. This is wasted work, but the waste is bounded (load-time only, not per-frame) and the cost of eliminating it requires rewriting the 2D painter.
2. **The fix is mechanical, not conceptual.** Once `gerber_geom` produces polygons for every layer type the 2D view currently shows (copper, mask, silkscreen, paste, drills), a custom egui painter consuming those polygons replaces `GerberRenderer::paint_layer()` at a single call site. `gerber_viewer` is then dropped from the dependency tree entirely, and the pipeline diagram above becomes the literal code rather than the intended code. Tracked as Phase N in Section 5.

Future Work Retiring `gerber_viewer` is blocked on `gerber_geom` covering every primitive the 2D canvas renders today: flashed apertures (circle, rect, obround, polygon, macro), drawn strokes with aperture widths, regions, and arcs. Phase-3 scope only handles the subset needed for board outlines (strokes, arcs, regions). Copper and mask support land in later phases; once they exist, the 2D swap is a single egui painter and a `Cargo.toml` edit.

3. Stage-by-Stage Reference

This section describes each pipeline stage in the order data flows through it. Each stage has a clear input contract, a clear output contract, and a single module responsible for implementation.

3.1. Stage 1: Gerber File Discovery

Input: a directory on disk containing a project's gerber output (typically produced by `kiCad-cli pcb export gerbers`).

Output: a populated `layer_store::LayerStore` where each `PcbLayer` carries the file path to its source gerber, a `LayerType` classification (Copper / Soldermask / MechanicalOutline / Drill / etc.), and a parsed `gerber_viewer::GerberLayer` for 2D rendering.

Implementation: `crate::layer_store::load_from_directory()` + `crate::layer_store::detection`. The detection module maps filename patterns (e.g. `*-Edge_Cuts.gbr` → `LayerType::MechanicalOutline`) to layer types. This mapping is the one piece of KiCad-specific convention in the pipeline; extending to other vendors means adding more filename patterns.

3.2. Stage 2: Gerber Document Parse

Input: a path to a single gerber file.

Output: a `gerber_parser::GerberDoc` containing the command stream, aperture table, units, and coordinate format.

Implementation: `gerber_parser::parse(BufReader)`. Returns `Result<GerberDoc, (GerberDoc, ParseError)>`; even on the error arm, the partial document is usable — failures are typically trailer issues that don't invalidate the geometry commands already parsed.

Design Decision Coordinates in `GerberDoc` are stored as `CoordinateNumber { nano: i64 }`, a fixed 6-decimal-place integer representation. The `From<CoordinateNumber>` for `f64` conversion gives the value in the file's declared unit (mm or inches). Modern Gerber X3 files are always absolute (`G91` relative mode was removed from the 2014 spec), so the consumer does not need to track absolute-vs-relative state — only the modal carry of the current X / Y between consecutive operations.

3.3. Stage 3: Geometry Extraction

Input: a `GerberDoc`.

Output: a set of line segments (from drawn strokes) and closed polygons (from G36 / G37 regions), all in mm, in the gerber's original coordinate space.

Implementation: `crate::gerber_geom::walk_commands()`. The walker is a modal state machine with four pieces of state:

- `pos_mm` — the current pen position (mm, the *modal carry* the parser does not maintain for the consumer).
- `mode` — active interpolation mode (`Linear` / `ClockwiseCircular` / `CounterclockwiseCircular`), set by G01 / G02 / G03.
- `in_region` — whether we are inside a G36 / G37 region block. Controls whether `Interpolate` ops emit strokes (outside a region) or region-boundary vertices (inside).
- `region_pts` — the vertex list accumulated for the currently-open region; flushed to the output on G37.

Every coordinate reference goes through `apply_coords(state, coords)`, which reads `Option<CoordinateNumber>` from the gerber and substitutes `state.pos_mm` for the missing component. This is the single chokepoint that guarantees no silently-broken geometry from omitted X or Y values.

Design Decision Arc flattening uses a sagitta-based tolerance: `steps = ceil(sweep · r / sqrt(8 · tol))` with `tol = 50 μm`, clamped to `[4, 256]`. Full-circle arcs (identical start/end under G75 multi-quadrant mode) are detected and given a full 2π sweep; without this detection, a closed circle would collapse to a single chord.

3.4. Stage 4: Contour Stitching

Input: a flat list of `Segment { a, b }` line pairs.

Output: a list of closed contours (`Vec<Vec<Point2>>`), each one a sequence of vertices walking a closed loop.

Implementation: `crate::gerber_geom::stitch_segments()`. Builds an endpoint-to-segment adjacency map (keyed on quantised 1 μm points to survive float noise), then greedily walks from each unused segment until it closes back on its starting vertex. Segments that don't participate in a closed loop — typically numerical noise at the ends of a trace — are discarded.

Future Work The stitcher assumes every stroke participates in a closed loop. For layers where strokes are meaningful as *open paths* (e.g. silkscreen text, unenclosed fiducial

marks), a separate extraction pass will be needed that emits stadium-shaped polygons per stroke rather than attempting to close loops. Phase-3 scope only needs Edge.Cuts, which is always closed.

3.5. Stage 5: Tessellation

Input: a list of closed contours (outer boundary + cutouts, unordered).

Output: a triangle mesh (`mesh_vertices_2d: Vec<[f32; 2]>`, `mesh_indices: Vec<u32>`) in the gerber's original coordinate space.

Implementation: `lyon::tessellation::FillTessellator` with `FillRule::EvenOdd`. Contours are added to a single path; nested contours (e.g. a cutout inside the board outline) are punched through automatically by the even-odd rule — no manual winding management required.

3.6. Stage 6: World-Space Transform

Input: mesh in gerber-native coordinates (origin wherever the EDA tool placed it; Y conventionally points down in gerber coordinate system).

Output: mesh in world coordinates where the board's **center** sits at `(0, 0)` and Y points up (matching a top-down 3D camera).

Implementation: `x' = x - bbox.center.x`; `y' = bbox.center.y - y`, applied inline at the end of the tessellation stage. The gerber-tool's arbitrary origin (e.g. `alpha_filter` board's `(2995, 0)` lower-left) disappears here; the 3D scene is always framed relative to the board itself. Centering at the board's centroid (rather than its lower-left corner) means the default orbit camera — which looks at the world origin — frames the board without needing a pan target, and zoom-to-region / auto-fit math stays symmetric.

3.7. Stage 7: GPU Upload and Render

Input: world-space mesh.

Output: pixels.

Implementation: `crate::render3d::ColoredMesh::upload()` uploads the mesh into a VBO once at project load; per-frame, `crate::panels::gerber_view_3d` binds the VBO and issues a draw call through the `UnlitProgram` shader with a per-frame MVP uniform from `crate::render3d::Camera::mvp(viewport)`. Depth test is enabled with `LEQUAL`; depth buffer is explicitly requested as 24-bit in `NativeOptions` (without this, the default `depth_buffer: 0` silently disables all depth-test behaviour).

4. Viewer 3D Features

The 3D viewer panel (`crate::panels::gerber_view_3d`) hosts a ribbon + canvas that surfaces the interactive features described in this section. Features are written down here so the UX surface of the 3D view has a single reference independent of the code that happens to implement it today.

Each subsection names its current status: **implemented** (ships on the current branch), **planned** (roadmap item, not yet wired), or **partial** (partially working; follow-up work enumerated inline).

4.1. Ground Grid

Status: implemented.

A line-primitive XY grid at `Z = 0`, intended to give spatial context and a size reference. The grid re-sizes and re-steps whenever a new board loads.

- **Step selection.** A ribbon `ComboBox` offers `Auto` plus a natural-step list in the active display unit: 20 / 50 / 100 / 250 / 500 / 1000 mils under mils mode, or 0.1 / 0.25 / 0.5 / 1 / 2.5 / 5 / 10 / 25 / 50 / 100 mm under mm mode. `Auto` picks the step yielding 20 cells across the board's largest dimension, in whichever unit is active.
- **Unit persistence.** Manual picks are stored in mm so toggling the display unit doesn't drift the world-space grid; the ribbon label translates on the fly.
- **Visibility toggle.** Toggle button in the ribbon labelled `Grid`; also bound to the `G` hotkey when the pointer is over the 3D canvas. Hiding the grid leaves the axes gizmo and the board visible.
- **Extent.** Grid half-extent is `max(board_max_dim × 0.75, step × 3)` — comfortably past the board edges so the board corners sit well inside the grid rather than flush with its border.

4.2. Zoom to Region

Status: implemented.

Right-mouse-drag on the canvas draws a yellow selection rectangle. On release, the camera pans and zooms so the selection fills the viewport.

- The two screen-space corners of the selection are un-projected onto the `Z = 0` world plane via `render3d::unproject_to_z0(mvp_inverse, rect, pixel)`. That function shoots a ray from the near clip plane to the far clip plane through the given screen pixel and intersects it with the board plane, so the pan target is correct under any camera tilt — not just top-down.

- Camera `target` (the orbit pivot) is set to the midpoint of the un-projected box; zoom is sized from the box's larger dimension via the same `fit_to_bbox` used for auto-fit on load.
- Sub-8-pixel drags are rejected as accidental clicks.

4.3. Flip

Status: planned.

A one-keystroke flip between viewing the top and bottom of the board. Intended binding: `F` hotkey (cursor over canvas). Implementation will add a 180° yaw to `camera.rotation` about the world Y axis, leaving `target` and `zoom` untouched so the framed region stays the same — just viewed from the opposite side.

4.4. Measure

Status: planned.

3D ruler tool for reading distances between two points on the board. Intended interaction: click to place the first endpoint, drag or click again to place the second; a measurement line + distance label appear in the active display unit (mm or mils). Endpoints will be un-projected to `Z = 0` using the same helper zoom-to-region uses, so the ruler lives on the board plane and reads physical distances regardless of camera tilt. Parallel to the 2D-gerber ruler feature already in `SharedServices` (see `ruler_start` / `ruler_end`).

4.5. Hotkeys

Several keys mean different things in the 2D gerber view and the 3D view — `F` flips top/bottom in both, `R` rotates in both, `M` is the ruler/measure toggle in both. Without scoping, hitting `F` with the 3D tab focused would silently flip the 2D gerber behind it.

Scoping rule. Hotkeys are routed to the active tab via `egui_citizen`. Each dockable panel is a *citizen* with a one-hot active bit; `TabViewer::on_tab_button` calls `Dispatcher::activate(citizen_id)` when the user clicks a tab, which atomically sets that citizen's `active` flag to `true` and clears every other citizen's flag. The global hotkey dispatcher in `CopperForgeApp::update` reads `dispatcher.get(citizen_id).active.get()` and gates each handler accordingly — 2D handlers fire only when the 2D citizen is active, 3D handlers fire only when the 3D citizen is active.

One additional rule: `G` (grid toggle) is scoped more tightly — it requires pointer-hover over the 3D canvas, not just 3D-tab-active. This is because `G` is harmless to misfire (toggling the grid is easily reversed) and the tighter scoping lets the user keep the 3D tab focused while the mouse is parked elsewhere without accidentally losing the grid.

Key	Action	Status
G	Toggle ground grid visibility	Implemented
Double-click left	Restore default view + fit to board	Implemented
F	Flip top/bottom view	Planned (see Section 4.3)
R	Rotate board 90° in-plane	Planned
M	Enter measure mode (see Section 4.4)	Planned
Mouse wheel	Zoom in / out	Implemented
Left-drag	Orbit camera	Implemented
Right-drag	Zoom to region (see Section 4.2)	Implemented

4.6. 3D Gizmo

Status: implemented.

A line-primitive axis triad at world origin — red **X**, green **Y**, blue **Z** — lifted a hair above the grid plane (`z_base = 0.001`) so the axis lines win the depth test over the grid center-lines. Axis length scales with the loaded board: `axes_len = max(max_dim × 0.15, 3.0)` mm, so a 30 mm board shows a 4.5 mm gizmo and a 300 mm board shows a 45 mm gizmo — always readable as a reference but never swamping the view.

HUD labels (**X**, **Y**, **Z**) are painted as egui 2D text in the matching axis colours at each tip. For each axis, the origin and tip are projected to screen space via `render3d::project(mvp, rect, world)` ; the label is pushed a fixed 14-pixel offset past the tip along the on-screen origin→tip direction, so the letter never sits on top of the coloured line. When an axis points directly at (or away from) the camera, the projection degenerates and the label is drawn at the tip instead.

5. Phased Rollout

The step-by-step development plan for the 3D viewer lives in `develop/task3d-plan.md`. As of this document's v0.1.0, phase status is:

Phase	Description	Status
1	Axes gizmo + mouse orbit + wheel zoom	Done
2	Ground grid, unit-aware ComboBox (deferred to Phase 3 panel)	Partial
3	Gerber-driven board outline (this FDD's primary scope)	In Progress
4	Gerber-driven copper layer extrusion	Planned
5	Gerber-driven soldermask layer	Planned
6	Drill file (Excellon) parsing for through-board cutouts	Planned
7	Retire <code>gerber_viewer</code> : 2D canvas consumes <code>gerber_geom</code> polygons directly; drop dependency	Planned
8	<code>LitProgram</code> for shaded component bodies	Planned
9	Component body loader (WRL first, STEP later)	Planned
10	Per-instance placement from <code>.kicad_pcb</code> footprints	Planned

5.1. Phase 3 Scope (current work)

1. Module `crate::gerber_geom` implementing Stages 3–6 of the pipeline.
2. `SharedServices::board_outline: Option<gerber_geom::OutlineData>` populated during gerber load.
3. `gerber_view_3d` panel renders the outline mesh at `Z = 0` with FR-4 colour.
4. Test case: load a KiCad gerber export and see a correctly-sized board outline with cutouts.

5.2. Deferred

- **Drill file handling.** Excellon `.drl` parsing is a separate file format. Will land as Phase 3b once the outline path is stable.
- **Vendor-agnostic filename detection.** Current `layer_store::detection` matches KiCad's naming convention. Altium / Eagle / EasyEDA conventions need adding.
- **Component 3D models.** Reserved for Phase 9; orthogonal to the gerber pipeline.
- **Retiring the legacy 2D renderer.** See Section 2.1. Tracked as Phase 7 in the table above.

6. References

- `develop/task3d-plan.md` — step-by-step implementation plan for the 3D viewer phases.
- `crates/copperforge-core/src/gerber_geom/mod.rs` — module-level documentation for the geometry extraction stages described in Section 3.
- *alumina-interface* (timschmidt/alumina-interface) — reference Rust/eframe/glow renderer embedding pattern used by `render3d`.
- Gerber Format Specification, Revision 2022.11 — authoritative reference for the command stream semantics.